

CPPopt_realtime1

June 11, 2026

```
[1]: # =====  
# SECTION 1 : IMPORT LIBRARIES  
# =====  
  
import pandas as pd  
import numpy as np  
  
from scipy.stats import pearsonr  
from scipy.optimize import curve_fit  
  
from collections import deque  
from datetime import datetime  
  
import matplotlib.pyplot as plt  
from IPython.display import clear_output  
  
import warnings  
warnings.filterwarnings("ignore")
```

```
[2]: # =====  
# SECTION 2 : LOAD LIVE DATA  
# =====  
  
LIVE_FILE = "live_data.csv"
```

```
[3]: # =====  
# SECTION 3 : SYSTEM PARAMETERS  
# =====  
  
# Physiological calculations  
  
PRX_WINDOW_SAMPLES = 60      # 5 min window  
  
CPP_BIN_WIDTH = 5  
  
CPP_MIN = 40  
CPP_MAX = 120
```

```

CPP_OPT_HISTORY = 240          # 4 hr

MIN_R2 = 0.20

MIN_CPP_RANGE = 10

MIN_BINS_REQUIRED = 5

MAX_ALLOWED_GAP_SEC = 10

```

```

[4]: # =====
# SECTION 4 : DATA BUFFERS
# =====

map_buffer = deque(maxlen=PRX_WINDOW_SAMPLES)
icp_buffer = deque(maxlen=PRX_WINDOW_SAMPLES)

prx_history = []
mean_cpp_history = []
prx_time_history = []

cpp_series = []
prx_series = []
cppopt_series = []

time_series = []

```

```

[5]: # =====
# SECTION 5 : STATUS VARIABLES
# =====

last_processed_row = 0

last_timestamp = None

current_clinical_minute = None

samples_in_current_minute = 0

current_prx = np.nan
current_cppopt = np.nan

latest_map = np.nan
latest_icp = np.nan
latest_cpp = np.nan

```

```
[6]: # =====
# SECTION 6 : CPP CALCULATION
# =====

def calculate_cpp(map_value, icp_value):

    return map_value - icp_value
```

```
[7]: # =====
# SECTION 7 : PRX CALCULATION
# =====

def calculate_prx(map_window, icp_window):

    if len(map_window) < PRX_WINDOW_SAMPLES:
        return np.nan

    try:
        prx, _ = pearsonr(map_window, icp_window)
        return prx

    except:
        return np.nan
```

```
[8]: # =====
# SECTION 8 : QUADRATIC FUNCTION
# =====

def quadratic(x, a, b, c):

    return a*x**2 + b*x + c
```

```
[9]: # =====
# SECTION 9 : R2 CALCULATION
# =====

def calculate_r2(y_actual, y_pred):

    ss_res = np.sum((y_actual - y_pred)**2)

    ss_tot = np.sum((y_actual - np.mean(y_actual))**2)

    if ss_tot == 0:
        return 0

    return 1 - (ss_res / ss_tot)
```

```

[10]: # =====
# SECTION 10 : CPPopt CALCULATION
# =====

def calculate_cppopt(prx_hist, cpp_hist):

    if len(prx_hist) < CPP_OPT_HISTORY:
        return np.nan

    prx_hist = np.array(prx_hist[-CPP_OPT_HISTORY:])
    cpp_hist = np.array(cpp_hist[-CPP_OPT_HISTORY:])

    bins = np.arange(
        cpp_hist.min(),
        cpp_hist.max() + CPP_BIN_WIDTH,
        CPP_BIN_WIDTH
    )

    bin_centers = []
    bin_prx = []

    for i in range(len(bins)-1):

        mask = (
            (cpp_hist >= bins[i]) &
            (cpp_hist < bins[i+1])
        )

        if np.sum(mask) > 0:

            bin_centers.append(
                (bins[i] + bins[i+1]) / 2
            )

            bin_prx.append(
                np.mean(prx_hist[mask])
            )

    if len(bin_centers) < MIN_BINS_REQUIRED:
        return np.nan

    bin_centers = np.array(bin_centers)
    bin_prx = np.array(bin_prx)

    try:

        popt, _ = curve_fit(

```

```

        quadratic,
        bin_centers,
        bin_prx
    )

    a, b, c = popt

    if a <= 0:
        return np.nan

    cppopt = -b/(2*a)

    if cppopt < CPP_MIN or cppopt > CPP_MAX:
        return np.nan

    fitted = quadratic(bin_centers, *popt)

    r2 = calculate_r2(
        bin_prx,
        fitted
    )

    if r2 < MIN_R2:
        return np.nan

    if (cpp_hist.max() - cpp_hist.min()) < MIN_CPP_RANGE:
        return np.nan

    return cppopt

except:
    return np.nan

```

```

[11]: # =====
# SECTION 11 : GAP DETECTION
# =====

def check_gap(prev_time, current_time):

    if prev_time is None:
        return False

    gap_sec = (
        current_time - prev_time
    ).total_seconds()

    return gap_sec >= MAX_ALLOWED_GAP_SEC

```

```
[12]: # =====
# SECTION 12 : LIVE PLOTTING
# =====

def plot_live():

    clear_output(wait=True)

    plt.figure(figsize=(14,8))

    plt.subplot(3,1,1)
    plt.plot(time_series, cpp_series)
    plt.title("CPP")

    plt.subplot(3,1,2)
    plt.plot(time_series, prx_series)
    plt.title("PRx")

    plt.subplot(3,1,3)
    plt.plot(time_series, cppopt_series)
    plt.title("CPPopt")

    plt.tight_layout()
    plt.show()
```

```
[ ]: # =====
# SECTION 13 : MAIN LOOP
# =====

while True:

    try:

        df = pd.read_csv(LIVE_FILE)

        if len(df) <= last_processed_row:
            continue

        new_rows = df.iloc[last_processed_row:]

        for _, row in new_rows.iterrows():

            timestamp = pd.to_datetime(
                row["TTDate"] + " " + row["TTTime"]
            )

            MAP = row["mean1"]
```

```

ICP = row["mean2"]

CPP = calculate_cpp(MAP, ICP)

latest_map = MAP
latest_icp = ICP
latest_cpp = CPP

gap_found = check_gap(
    last_timestamp,
    timestamp
)

if gap_found:

    cpp_series.append(np.nan)
    prx_series.append(np.nan)
    cppopt_series.append(np.nan)

    time_series.append(timestamp)

map_buffer.append(MAP)
icp_buffer.append(ICP)

minute_stamp = timestamp.floor("min")

if current_clinical_minute is None:
    current_clinical_minute = minute_stamp

if minute_stamp == current_clinical_minute:

    samples_in_current_minute += 1

else:

    if (
        samples_in_current_minute > 0
        and
        len(map_buffer) >= PRX_WINDOW_SAMPLES
    ):

        current_prx = calculate_prx(
            list(map_buffer),
            list(icp_buffer)
        )

        mean_cpp = np.mean(

```

```

        np.array(map_buffer)
        -
        np.array(icp_buffer)
    )

    prx_history.append(current_prx)

    mean_cpp_history.append(mean_cpp)

    prx_time_history.append(timestamp)

    current_cppopt = calculate_cppopt(
        prx_history,
        mean_cpp_history
    )

    else:

        current_prx = np.nan
        current_cppopt = np.nan

        current_clinical_minute = minute_stamp
        samples_in_current_minute = 1

    time_series.append(timestamp)

    cpp_series.append(CPP)

    prx_series.append(current_prx)

    cppopt_series.append(current_cppopt)

    last_timestamp = timestamp

last_processed_row = len(df)

plot_live()

print("="*50)

print("Time      :", timestamp)
print("MAP       :", latest_map)
print("ICP       :", latest_icp)
print("CPP       :", latest_cpp)

print("PRx       :", current_prx)

```



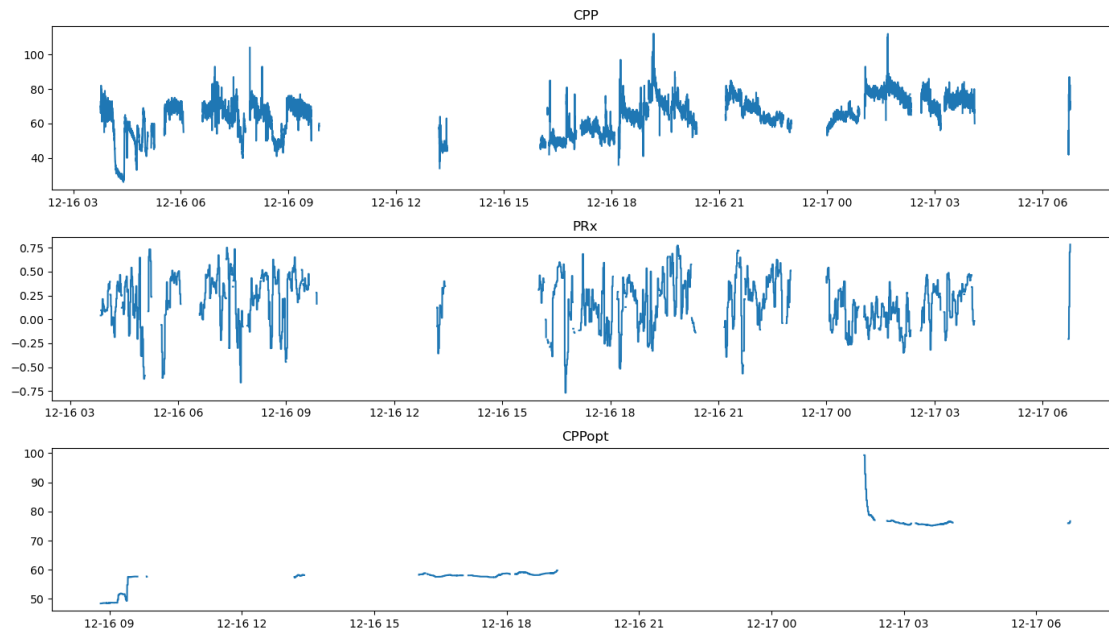
```

print("CPPopt :", current_cppopt)

except Exception as e:

print("Error:", e)

```



```

=====
Time      : 2016-12-17 06:46:06
MAP       : 111
ICP       : 29
CPP       : 82
PRx       : 0.7829381193631231
CPPopt    : 76.66769507221487

```

```

[ ]: if gap_sec > 10:

    print(
        f"DATA GAP DETECTED : "
        f"{gap_sec/60:.1f} minutes"
    )

```